

Migrate Content Slots from SFRA to Storefront Next

In Storefront Next, React components replace ISML templates. If you're migrating from Salesforce Storefront Reference Architecture (SFRA) and rely on content slots, use this workaround until the planned Embedded Content feature becomes available.

Content Slot Workaround Overview

This workaround lets you manage reusable HTML content, such as shipping policies, disclaimers, or promotional blocks, in Page Designer and embed it across your storefront. It provides:

- A "Content Slot Page" page type with a single `content` region.
- A "Markup Content" component type with a `markup` attribute for HTML.
- A React component that renders the markup using the

Contents

[Content Slot Workaround Overview](#)

[Step 1: Create the Markup Content Component](#)

[Step 2: Create the Content Slot Page Type JSON](#)

[Step 3: Create a Preview Route for Page Designer](#)

[Step 4: Generate and Deploy Metadata](#)

[Step 5: Set Up a Content Slot in Business Manager](#)

[Populate the Content](#)

[Embed Content in Your Storefront](#)

[Content Slot Workaround Limitations](#)

[Quick Reference](#)



This workaround is temporary. It will be replaced by the Embedded Content feature, which will provide dedicated APIs, a specialized Business Manager UI, and performance optimizations.

Step 1: Create the Markup Content Component

Create the markup content in

`src/components/markup-content/index.tsx`.

```
</> | [icon] | [icon]
1 import { Component
2 import { AttributeD
3 import { RegionDefi
4 import HtmlFragment
5 import type { Compo
6
7 @Component("markupC
8   name: "Markup Con
9   description: "Dis
10 })
11 @RegionDefinition([
12 export class Markup
13   @AttributeDefinit
14   markup?: string;
15 }
16
17 interface MarkupCon
18   markup?: string;
19   component?: Compo
20 }
21
22 export default func
23   const content = m
24
25   if (!content) ret
26
27   return <HtmlFragm
28 }
```

[Try for free](#)

Page Type JSON

Create the content slot page. Add the `contentSlotPage.json` file in `cartridges/app_storefrontnext_base/cartridge/experience/pages/` and add this content.

```
</> | |
1 {
2   "name": "Content
3   "description": "M
4   "arch_type": "hea
5   "region_definitio
6     {
7       "id": "conten
8       "name": "Cont
9       "description"
10      "max_componen
11    }
12  },
13  "supported_aspect
14  "route": "/conten
15 }
```

Step 3: Create a Preview Route for Page Designer

Content slot pages need a preview route to be editable in Page Designer's design mode. Create the `_app.content-slot.tsx` file in `src/routes/_app.content-slot.tsx` and add this content.

```
</> | |
1 import { Region } f
2 import { fetchPageW
3 import type { Load
4
5 export function loa
```



```
11  
12 export default func  
13   return (  
14     <div className=  
15       <Region page=  
16     </div>  
17   );  
18 }
```

Step 4: Generate and Deploy Metadata

Use these commands to generate and deploy the metadata.

```
</> | |  
1  pnpm generate:cartr  
2  pnpm push  
3  pnpm sfnext deploy-
```

Step 5: Set Up a Content Slot in Business Manager



Important

To be able to bind content assets to components in Page Designer, turn on a feature switch. In **Administration > Global Preferences > Feature Switches**, turn on **Enable Page Designer Data Binding**.

1. Create a content slot page in **Merchant Tools > Content >**



3. Add **Markup Content** components to the “content” region.
4. Configure markup by entering HTML directly or by using data binding to bind to Content Assets.
5. Assign a page ID (for example, `shipping-policy` or `legal-disclaimer`). You reference this ID when fetching the page.

Populate the Content

You can populate content in two ways.

Option 1: Direct HTML entry

Enter HTML directly in the Markup field of a Markup Content component. This approach is best for content that is specific to your Storefront Next storefront.

Option 2: Data binding to content assets

Bind the Markup field to a content asset’s HTML attribute. This approach is best for content that must be shared across multiple storefront technologies (SFRA and Storefront Next). Content assets are platform-agnostic, so you can update content once and have it reflected across all storefronts.

Embed Content in



`fetchPageWithComponentData` and the `Region` component to load and render content slot page data anywhere in your storefront.

```
</> | ☰ | 📄  
  
1 import { Region } from '@shopify/region-component';  
2 import { fetchPageWithComponentData } from '@shopify/hydrogen';  
3 import type { Loadable } from '@shopify/hydrogen';  
4  
5 export function loadPage(): Loadable {  
6   return {  
7     basket: fetchBasket(),  
8     // Load content slot data  
9     shippingInfo: fetchShippingInfo(),  
10  };  
11 }  
12  
13 export default function Page() {  
14   return (  
15     <div>  
16       <h1>Checkout</h1>  
17       { /* Your page content */ }  
18       { /* Embedded content slot */ }  
19       <aside className="region">  
20         <Region pageData={pageData} />  
21       </aside>  
22     </div>  
23   );  
24 }  
25 }
```

Content updates in Business Manager are reflected immediately without code changes.

Content Slot Workaround Limitations

- No dedicated APIs for content slot management—uses the same `fetchPageWithComponentData` as regular pages.
- No caching or CDN strategies specific to embedded content.
- Merchants must navigate to Page Designer to edit



page in Business Manager for content that isn't actually a full page.

When the “Embedded Content” feature ships, migrate your content slot pages to the new system and update component references to the new API.

Quick Reference

Component Checklist

- ☐ `@Component` decorator with `id`, `name`, `group`, `description`
- ☐ `@AttributeDefinition` for each configurable prop
- ☐ `@RegionDefinition` if component contains nested regions
- ☐ `export function fallback()` - lightweight loading state
- ☐ `export default` - main component
- ☐ `export const loader` - if data loading needed

Page Checklist

- ☐ `@PageType` decorator with `name`, `description`, `supportedAspectTypes`
- ☐ `@RegionDefinition`